



QTX-AH — Claude Code Session Prompts

Each section below is a **complete, self-contained prompt** ready to paste into a fresh Claude Code session. Copy everything from `--- START PROMPT ---` to `--- END PROMPT ---` for that feature.

Branch naming convention: `feat/<feature-slug>` — branch from `main`, implement, verify tests pass, then the audit agent (see the final section) reviews and merges.

Feature 1 — Retrain Regression Model with Longitudinal Features

--- START PROMPT ---

You are working on QTX-AH, a clinical intelligence platform for rehabilitation patients and

CURRENT STATE: The regression model (models/regression_xgb.joblib) achieves $R^2 = 0.022$. The

TASK: Implement Feature 1 – Retrain Regression Model with Longitudinal Features.

BRANCH: Create and work on branch feat/longitudinal-regression. Branch from main.

WHAT TO DO:

1. config/models.yaml – In the regression_composite.features list, add after the last rgn_
– session_number
– prior_avg_composite_improvement
– trend_tug_magnitude
2. scripts/18_scheduled_retrain.py – The REGRESSION_FEATURES list already contains these
– session_number: already in sessions table as s.session_number – just add it to the SQL
– prior_avg_composite_improvement: add as a window function:
COALESCE(AVG(s.composite_improvement) OVER (PARTITION BY s.patient_id ORDER BY s.session_number), 0.0) AS prior_avg_composite_improvement
– trend_tug_magnitude: add as a lateral join:
LEFT JOIN LATERAL (SELECT magnitude FROM patient_trends WHERE patient_id = s.patient_id) AS tug_trend
then COALESCE(tug_trend.magnitude, 0.0) AS trend_tug_magnitude
3. Run the retrain to validate it works:
PYTHONPATH=src:api DATABASE_URL=postgresql+psycopg2://qtx:secret@localhost:5432/qtxah
The script saves the new model only if CV metrics hold or improve vs retrain_state.json
4. tests/test_longitudinal_features.py (existing file) – add tests:
– Test that after retraining, the loaded model's feature_names_in_ includes all three new features
– Test _get_longitudinal_features() returns prior_avg_composite_improvement=0.0 for a patient with no prior data
– Test _get_longitudinal_features() returns a non-zero prior_avg when a patient has two or more sessions

VERIFY: Run PYTHONPATH=src:api .venv/bin/pytest tests/test_longitudinal_features.py -v and

COMMIT: Commit all changes with a clear message. Do not merge to main – leave on feat/longitudinal-regression

--- END PROMPT ---

Feature 2 — Per-Patient Bayesian Residual Correction

--- START PROMPT ---

You are working on QTX-AH, a clinical intelligence platform for rehabilitation patients and

CURRENT STATE: PredictionService._run() produces a raw XGBoost point estimate for composi

TASK: Implement Feature 2 – Per-Patient Bayesian Residual Correction.

BRANCH: Create and work on branch feat/bayesian-residual-correction. Branch from main.

WHAT TO DO:

1. api/services/prediction.py – Add this function after _get_longitudinal_features():

```
def _compute_patient_bias(patient_id, db) -> float:
    """Returns AVG(predicted - actual) from this patient's session history. 0.0 if <2 sessions"""
    from models.clinical import SessionPrediction, Session as ClinicalSession
    rows = (
        db.query(SessionPrediction.predicted_composite_improvement, ClinicalSession.composite_improvement)
        .join(ClinicalSession, SessionPrediction.session_id == ClinicalSession.id)
        .filter(
            SessionPrediction.patient_id == patient_id,
            SessionPrediction.predicted_composite_improvement.isnot(None),
            ClinicalSession.composite_improvement.isnot(None),
        ).all()
    )
    if len(rows) < 2:
        return 0.0
    residuals = [float(pred) - float(actual) for pred, actual in rows]
    return sum(residuals) / len(residuals)
```

2. api/services/prediction.py – In PredictionService._run(), after computing predictions[

```
    bias = _compute_patient_bias(patient.id, self._db)
```

```
    if bias != 0.0:
```

```
        predictions["predicted_composite_improvement"] = predictions["predicted_composite_improvement"] - bias
        predictions["bias_correction_applied"] = round(bias, 4)
```

3. scripts/23_migrate_add_bias_correction.py – New migration script:

```
ALTER TABLE session_predictions ADD COLUMN IF NOT EXISTS bias_correction NUMERIC(7,4)
```

```
Run it: DATABASE_URL=postgres://qtx:secret@localhost:5432/qtxah PYTHONPATH=. python scripts/23_migrate_add_bias_correction.py
```

4. `api/models/clinical.py` – Add to `SessionPrediction` class:
`bias_correction: Mapped[float | None] = mapped_column(Numeric(7, 4), nullable=True)`
5. `api/services/prediction.py` – In the `SessionPrediction` row creation, add:
`bias_correction=predictions.get("bias_correction_applied"),`
6. `api/routers/sessions.py` – In the `GET /api/patient/{sn}/predictions/latest` response dict
`"bias_correction": float(row.bias_correction) if row.bias_correction is not None else None`
7. `web/components/clinical/PredictionChips.tsx` – If `bias_correction` is non-null in the API response, show it.
8. `tests/test_prediction_service.py` – Add:
 - Test `_compute_patient_bias` returns `0.0` when `<2` sessions
 - Test `_compute_patient_bias` correctly averages residuals across multiple sessions
 - Test that when `bias=0.05`, the final `predicted_composite_improvement` equals `raw - 0.05`
 - Test that `bias_correction_applied` is stored in `predictions` dict

VERIFY: `PYTHONPATH=src:api .venv/bin/pytest tests/ -v` – all tests must pass.

COMMIT and leave on `feat/bayesian-residual-correction`.

--- END PROMPT ---

Feature 3 — Proactive Anomaly Detection (Extend Existing Implementation)

--- START PROMPT ---

You are working on QTX-AH, a clinical intelligence platform for rehabilitation patients and

CURRENT STATE: Anomaly detection is ALREADY PARTIALLY IMPLEMENTED. The files `api/services/`

TASK: Implement Feature 3 – complete and wire up the anomaly detection system.

BRANCH: Create and work on branch `feat/anomaly-detection`. Branch from `main`.

WHAT TO DO:

1. Read `api/services/anomaly.py` carefully to understand the current implementation.
2. `api/services/anomaly.py` – In `_detect_flags()`:
 - a. Fix `pain_worsened`: change condition from `float(session.post_vas) > float(session.pre_vas)`
 - b. Add Rule 5 – `HIGH_DROPOUT_RISK`: if `predictions` is not `None` and `float(predictions.get('high_dropout_risk')) > 0.74`
3. `api/services/anomaly.py` – Update `_build_warning_prompt()` to include `dropout_probabilities`
4. Read `web/components/clinical/AITab.tsx`. Find where insights are rendered. If `AnomalyWarningCard`:
 - a. Import `AnomalyWarningCard` from `'../clinical/AnomalyWarningCard'`
 - b. Add a fetch for `GET /api/patient/{sn}/anomalies/latest` (this endpoint exists in `api`)
 - c. Render `AnomalyWarningCard` above the session insight cards when a warning exists for the session
5. `web/lib/api.ts` – Add function `getLatestAnomaly(sn: string)` that calls `GET /api/patient/{sn}/anomalies/latest`
6. `web/lib/types.ts` – Add type `AnomalyWarning { session_number: number; content: string; }`
7. `tests/test_anomaly_detection.py` – Create new test file:
 - Test each rule fires independently with crafted mock data
 - Test `pain_worsened` does NOT fire when `post_vas = pre_vas + 0.5` (below `1.0` threshold)
 - Test `pain_worsened` DOES fire when `post_vas = pre_vas + 1.5`
 - Test `high_dropout_risk` fires at `0.80` and does NOT fire at `0.74`
 - Test `responder_mismatch` fires when `composite_improvement < 0` AND `responder_probabilities`
 - Test `fall_risk_unregistered` fires when `post_tug_s > 12.0` and `has_fall_risk` is `False`
 - Test `check_and_warn` returns `None` when no flags fire (no DB write)
 - Test `check_and_warn` in stub mode (no API key) returns `STUB_RESPONSE` and writes a row

VERIFY: `PYTHONPATH=src:api .venv/bin/pytest tests/ -v` – all tests pass.

COMMIT and leave on feat/anomaly-detection.

--- END PROMPT ---

Feature 4 — Treatment Plan Generation

--- START PROMPT ---

You are working on QTX-AH, a clinical intelligence platform for rehabilitation patients and

CURRENT STATE: InsightService generates session summaries and Q&A responses. It has no tr

TASK: Implement Feature 4 – Treatment Plan Generation.

BRANCH: Create and work on branch feat/treatment-plan. Branch from main.

WHAT TO DO:

1. `api/services/insight.py` – Add a new prompt template constant `_TREATMENT_PLAN_TEMPLATE` and
– Accept patient phenotype flags (`has_frailty`, `has_diabetes`, `has_neurological`, `has_stroke`)
– Accept the latest session data and all trend directions/magnitudes
– Accept the ML prediction signals (`predicted_composite_improvement`, `responder_probability`)
– Accept an optional `clinician_focus` string and `plan_sessions` int (default 4)
– Call `_call_claude()` with `max_tokens=1500` (longer than usual – plans are detailed)
– Persist the result as `insight_type="treatment_plan"` via `_save_insight()`
– Return the generated text (or `STUB_RESPONSE` if no API key)

The generated plan must use this exact structure:

****Session Focus:**** [one sentence]

****Session-by-Session Plan:**** bullet per session

****Key Metrics to Monitor:**** [metric] – target: [MCID-based value]

****Risk Flags:**** [phenotype-specific cautions]

****Dosage Recommendation:**** [from model signal]

2. `api/routers/plan.py` – New file. Create POST `/api/patient/{sn}/suggest_plan` endpoint:
– Pydantic request body: `{ session_focus: str | None = None, plan_sessions: int = 4 }`
– Load patient from DB, 404 if not found
– Load all sessions ordered by `session_number`, all trends, latest `SessionPrediction` row
– Build timeline and predictions dicts (same pattern as `sessions.py` and `ask.py`)
– Call `InsightService(db).generate_treatment_plan(...)`
– `db.commit()`
– Return `{ plan: str, generated_at: iso8601 }`
3. `api/main.py` – Import plan router and register: `app.include_router(plan_router, prefix="/api")`
4. `web/components/clinical/PlanCard.tsx` – New component. Accepts `{ plan: string, generated_at: string }`

5. web/lib/api.ts – Add suggestPlan(sn: string, body: { session_focus?: string, plan_sessions?: number })
6. web/lib/types.ts – Add PlanRequest and TreatmentPlanResponse types
7. web/components/clinical/AITab.tsx – Add a "Generate Treatment Plan" button. When clicked, it should call the suggestPlan API and display the treatment plan.
8. tests/test_plan_api.py – New test file:
 - Test POST /api/patient/{sn}/suggest_plan returns 200 with plan string in stub mode (no db call)
 - Test 404 for unknown patient SN
 - Test that a treatment_plan row is written to patient_insights table after the call
 - Test plan_sessions=6 is accepted without error

VERIFY: PYTHONPATH=src:api .venv/bin/pytest tests/ -v – all tests pass.

COMMIT and leave on feat/treatment-plan.

--- END PROMPT ---

Feature 5 — Cohort Percentile Ranks (Per-Metric)

--- START PROMPT ---

You are working on QTX-AH, a clinical intelligence platform for rehabilitation patients and

CURRENT STATE: GET /api/patient/{sn}/benchmark exists in api/routers/benchmark.py and returns

TASK: Implement Feature 5 – Cohort Percentile Ranks (per-metric).

BRANCH: Create and work on branch feat/cohort-percentiles. Branch from main.

WHAT TO DO:

1. api/routers/benchmark.py – Replace the existing simple query with a richer query that

```
{
  "cohort": "...",
  "cohort_n": int,
  "cohort_percentile": int, // keep existing field for backwards compat
  "benchmarks": [
    { "metric": "composite_improvement", "patient_value": float, "percentile": float (0-100) },
    { "metric": "tug_change_pct", ... },
    { "metric": "sst_change_pct", ... },
    { "metric": "sppb_change", ... },
    { "metric": "vas_change", ... },
    { "metric": "normal_gs_change_pct", ... },
    { "metric": "fast_gs_change_pct", ... }
  ]
}
```

Use a single CTE with window functions. The key logic:

- composite_improvement: PERCENT_RANK() ORDER BY value ASC (higher = better)
- tug_change_pct: ORDER BY value DESC (more negative = bigger improvement = better rank)
- sst_change_pct: ORDER BY value DESC (same as TUG)
- sppb_change: ORDER BY value ASC (higher = better)
- vas_change: ORDER BY value DESC (more negative = less pain = better)
- normal_gs_change_pct: ORDER BY value ASC (higher = better)
- fast_gs_change_pct: ORDER BY value ASC (higher = better)

Only include a metric in the response if the patient has a non-null value for it.
Filter to QTX sessions only (ingested_from NOT ILIKE '%raise%').

Helper function for ordinal suffix (1→"1st", 2→"2nd", 3→"3rd", N→"Nth"):

```
def _ordinal(n: int) -> str:
    if 11 <= (n % 100) <= 13: return f"{n}th"
    return f"{n}{'th','st','nd','rd','th'}[min(n%10,4)]"
```

2. web/lib/types.ts – Add:

```
export type BenchmarkMetric = { metric: string; patient_value: number; percentile: number }
export type BenchmarkResult = { cohort: string; cohort_n: number; cohort_percentile: number }
```

3. web/lib/api.ts – Update getPatientBenchmark(sn) to return BenchmarkResult

4. web/components/clinical/BenchmarkCard.tsx – New component. Renders each metric as a row.
Friendly metric labels: composite_improvement → "Overall Improvement", tug_change_pct → "Tug Change %"

5. web/components/clinical/AITab.tsx – Import and render BenchmarkCard. Fetch benchmark data

6. tests/test_benchmark_api.py – New or extend existing test file:

- Test response includes benchmarks array with at least one metric
- Test percentile values are between 0.0 and 1.0
- Test cohort_percentile field is preserved in response
- Test 404 for unknown patient SN
- Test patient with no session data returns cohort_percentile: null

VERIFY: PYTHONPATH=src:api .venv/bin/pytest tests/ -v – all tests pass.

COMMIT and leave on feat/cohort-percentiles.

--- END PROMPT ---

Feature 6 — Wire Wearable Cadence into Fall Risk Context

--- START PROMPT ---

You are working on QTX-AH, a clinical intelligence platform for rehabilitation patients and

CURRENT STATE: `api/services/wearable_features.py` computes `wearable_cadence_avg_30d` (average

TASK: Implement Feature 6 — Wire Wearable Cadence into Fall Risk Context.

BRANCH: Create and work on branch `feat/wearable-cadence`. Branch from `main`.

WHAT TO DO:

1. Read `api/services/wearable_features.py` to understand the exact function signature and
2. `api/routers/sessions.py` — Before calling `InsightService.generate_session_insight()`, add
Import the `wearable` function at the top of `sessions.py`.
3. `api/services/anomaly.py` — Add a fifth rule `LOW_CADENCE_RISK`:
 - Add parameter `wearable_feats: dict | None = None` to `check_and_warn()` signature
 - In `_detect_flags()`, if `wearable_feats` is not `None`: `cadence = wearable_feats.get("cadence_avg_30d")`
 - Update `_build_warning_prompt()` to include `cadence` in the `measurements` block when present
4. `api/services/insight.py` — In `_SYSTEM_PROMPT`, add one sentence: "Walking cadence (wearable)"
5. `api/routers/sessions.py` — Update the `check_and_warn()` call to pass `wearable_feats=wearable_feats`
6. `tests/test_wearable_features.py` or `tests/test_anomaly_detection.py` — Add:
 - Test `LOW_CADENCE_RISK` fires when `cadence_avg_30d = 75`
 - Test `LOW_CADENCE_RISK` does NOT fire at 85
 - Test `LOW_CADENCE_RISK` does NOT fire (no exception) when `cadence_avg_30d` is `None`
 - Test `check_and_warn` still works when `wearable_feats=None` (backwards compat)

VERIFY: `PYTHONPATH=src:api .venv/bin/pytest tests/ -v` — all tests pass.

COMMIT and leave on `feat/wearable-cadence`.

--- END PROMPT ---

Feature 7 — Secrets Rotation and Environment Hardening

--- START PROMPT ---

You are working on QTX-AH, a clinical intelligence platform for rehabilitation patients and

CURRENT STATE: The .env file contains live API keys but may have been accidentally committed

TASK: Implement Feature 7 – Secrets Rotation and Environment Hardening.

BRANCH: Create and work on branch feat/secrets-hardening. Branch from main.

IMPORTANT: Do NOT rotate or modify actual API key values in .env – the user will do that

WHAT TO DO:

1. Check git history for .env: run `git log --all --full-history --oneline -- .env`
If .env appears in git history, output instructions for the user to clean it with BFG
2. Verify .gitignore contains .env, .env.local, and .env.*.local. If any are missing, add
3. Create .env.example (safe to commit – only placeholder values):
QTX_API_KEY=<generate: python3 -c "import secrets; print(secrets.token_hex(32))">
QTX_ADMIN_KEY=<must differ from QTX_API_KEY>
DATABASE_URL=postgresql+psycopg2://qtx:secret@localhost:5432/qtxah
VOYAGE_API_KEY=<from dash.voyageai.com>
ANTHROPIC_API_KEY=<from console.anthropic.com>
LOG_LEVEL=INFO
RETRAIN_THRESHOLD=50
CALIBRATION_DRIFT_THRESHOLD=0.30
CALIBRATION_MIN_COHORT_N=20
ALLOWED_ORIGINS=http://localhost:3000
TERRA_WEBHOOK_SECRET=
TERRA_DEV_ID=
TERRA_API_KEY=
4. Create .git/hooks/pre-commit (executable) that blocks commits of .env files and scans for
#!/bin/bash
Block .env file commits
if git diff --cached --name-only | grep -qE '^(^|/)\.env(\$|\.)'; then
 echo "ERROR: Refusing to commit .env file. Remove it from staging first."
 exit 1

```
fi
# Block Anthropic key patterns
if git diff --cached | grep -qE 'sk-ant-[a-zA-Z0-9\-\_]{40,}'; then
    echo "ERROR: Possible Anthropic API key in staged changes."
    exit 1
fi
Make it executable: chmod +x .git/hooks/pre-commit
```

5. Update HANDOFF.md – In the "Environment files" section, add a WARNING block stating all

6. Update README.md – Add a "Security" section near the top noting the .env.example file and

NOTE: Do not add ALLOWED_ORIGINS to api/main.py in this task – that is Feature 8.

VERIFY: Confirm .gitignore is correct. Confirm .env.example exists. Confirm pre-commit hook .

COMMIT and leave on feat/secrets-hardening.

--- END PROMPT ---

Feature 8 — CORS Hardening

--- START PROMPT ---

You are working on QTX-AH, a clinical intelligence platform for rehabilitation patients and

CURRENT STATE: api/main.py has CORSMiddleware hardcoded to allow_origins=["http://localhost:3000"]

TASK: Implement Feature 8 — CORS Hardening.

BRANCH: Create and work on branch feat/cors-hardening. Branch from main.

WHAT TO DO:

1. api/main.py — Replace the hardcoded CORS origins list with an environment-driven approach
import os
_raw_origins = os.environ.get("ALLOWED_ORIGINS", "http://localhost:3000")
_allowed_origins = [o.strip() for o in _raw_origins.split(",") if o.strip()]
Then use allow_origins=_allowed_origins in the CORSMiddleware call.
This must be computed AFTER load_dotenv() is called (which is already at the top of main.py)
2. .env — Add line: ALLOWED_ORIGINS=http://localhost:3000
(If .env does not exist or you cannot read it, add a comment to .env.example instead and create .env)
3. HANDOFF.md — In Local Dev Setup > Environment files section, document ALLOWED_ORIGINS. Make sure to include the .env file and how to use it.
4. tests/test_middleware.py (existing) — Add three tests:
 - Test that a request with Origin: http://localhost:3000 passes when ALLOWED_ORIGINS=http://localhost:3000
 - Test that setting ALLOWED_ORIGINS to two values (comma-separated) makes both work
 - Test that the middleware correctly reads the env var (mock os.environ.get and verify it)

VERIFY: PYTHONPATH=src:api .venv/bin/pytest tests/ -v — all tests pass, especially test_middleware.py

COMMIT and leave on feat/cors-hardening.

--- END PROMPT ---

Feature 9 — Background Task Queue for Retrain Jobs

--- START PROMPT ---

You are working on QTX-AH, a clinical intelligence platform for rehabilitation patients and

CURRENT STATE: RetrainService.check_and_trigger() and CalibrationService.check_and_trigger()

TASK: Implement Feature 9 – Background Task Queue for Retrain Jobs.

BRANCH: Create and work on branch feat/background-tasks. Branch from main.

WHAT TO DO:

1. Read api/services/retrain.py carefully to understand check_and_trigger() current signature
2. Read api/services/calibration.py to understand CalibrationService.check_and_trigger() c
3. api/routers/sessions.py – In the create_session handler, import BackgroundTasks from fa
from fastapi import APIRouter, BackgroundTasks, Depends, HTTPException
def create_session(sn, body, background_tasks: BackgroundTasks, db: DBSession = Depends
Then pass background_tasks to both service calls.
4. api/services/retrain.py – Update check_and_trigger() to accept background_tasks parameter
5. api/services/calibration.py – Apply the same pattern: check_and_trigger() accepts backg
6. Ensure the retrain logic (subprocess call, state file update, reload-models call) stays
7. tests/test_retrain_service.py (existing) – Add:
 - Test that when trigger condition is met, background_tasks.add_task is called (mock B
 - Test that add_task is NOT called when threshold not reached
 - Test that _spawn_retrain_subprocess logs an error when the script exits non-zero
8. tests/test_calibration_service.py (existing) – Add similar BackgroundTasks mock tests.

VERIFY: PYTHONPATH=src:api .venv/bin/pytest tests/ -v – all tests pass.

COMMIT and leave on feat/background-tasks.

--- END PROMPT ---

Feature 10 — Session Preparation Workflow

--- START PROMPT ---

You are working on QTX-AH, a clinical intelligence platform for rehabilitation patients and

CURRENT STATE: The "Prepare Session" button exists in web/components/clinical/QAPanel.tsx

TASK: Implement Feature 10 – Session Preparation Workflow.

BRANCH: Create and work on branch feat/session-prep. Branch from main.

WHAT TO DO:

1. api/services/insight.py – Add `_PRE_SESSION_BRIEF_TEMPLATE` constant and `generate_pre_session_brief` function:
 - `**Last Session Summary**` (2 sentences max) – what happened in the most recent session
 - `**Current Trajectory**` (2 sentences max) – which metrics are improving/plateauing/declining
 - `**Watch For Today**` (3 bullet points) – most important things to observe in today's session
 - `**Suggested Questions**` (2 bullet points) – specific questions to ask the patient at session startCall `_call_claude()` with `max_tokens=512`. Persist as `insight_type="pre_session_brief"` via `insights.save_insight()`.
2. api/routers/ask.py – Add POST `/api/patient/{sn}/prepare_session` endpoint:
 - Load patient (404 if not found)
 - Find the latest `session_number` for this patient
 - Check if a `pre_session_brief` insight already exists for this `session_number` (`SELECT * FROM insights WHERE patient_sn={sn} AND session_number={sn} AND insight_type='pre_session_brief'`)
 - If it exists: return `{"brief": existing.content, "cached": True, "created_at": iso8601_timestamp}`
 - If not: build timeline (same pattern as the existing ask endpoint), call `generate_pre_session_brief`
 - Support optional `?force=true` query param to skip the cache check and regenerate
3. web/lib/api.ts – Add `prepareSession(sn: string, force?: boolean)` function calling POST `/api/patient/{sn}/prepare_session`
4. web/components/clinical/QAPanel.tsx – Update the "Prepare Session" button:
 - Change it to call `prepareSession(sn)` instead of the generic ask endpoint
 - Show a loading spinner while the API call is in flight
 - When the response arrives, render the brief in a distinct pre-session panel (above the session notes)
 - Show a small "Cached" badge if the response has `cached: true`
 - Show a "Refresh" link that calls `prepareSession(sn, true)` to force regeneration
5. tests/test_ask_api.py (existing) – Add:
 - Test POST `/api/patient/{sn}/prepare_session` returns 200 with brief and `cached: false`
 - Test second call returns `cached: true` without calling `InsightService` again
 - Test `?force=true` bypasses cache and calls `InsightService`

- Test 404 for unknown patient

VERIFY: PYTHONPATH=src:api .venv/bin/pytest tests/ -v - all tests pass.

COMMIT and leave on feat/session-prep.

--- END PROMPT ---

Feature 11 — Comparative Cohort Response Curves

--- START PROMPT ---

You are working on QTX-AH, a clinical intelligence platform for rehabilitation patients and

CURRENT STATE: The CohortsPage shows static breakdowns. MetricChart.tsx shows a patient's

TASK: Implement Feature 11 – Comparative Cohort Response Curves.

BRANCH: Create and work on branch feat/cohort-response-curves. Branch from main.

WHAT TO DO:

1. api/models/clinical.py – Add CohortResponseCurve ORM model:

```
class CohortResponseCurve(Base):
    __tablename__ = "cohort_response_curves"
    __table_args__ = (UniqueConstraint("grp_flag", "metric", "session_number", name="uq_response_curves_grp_metric_session"),)
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    grp_flag: Mapped[str] = mapped_column(String(50), nullable=False, index=True)
    metric: Mapped[str] = mapped_column(String(50), nullable=False)
    session_number: Mapped[int] = mapped_column(SmallInteger, nullable=False)
    p25: Mapped[float | None] = mapped_column(Numeric(8, 4), nullable=True)
    p50: Mapped[float | None] = mapped_column(Numeric(8, 4), nullable=True)
    p75: Mapped[float | None] = mapped_column(Numeric(8, 4), nullable=True)
    n: Mapped[int] = mapped_column(SmallInteger, nullable=False)
    computed_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
```

2. scripts/24_migrate_cohort_response_curves.py – Migration that creates the table and index.
CREATE TABLE IF NOT EXISTS cohort_response_curves (...) with the same columns as above.
CREATE UNIQUE INDEX uq_response_curves_grp_metric_session ON cohort_response_curves (grp_flag, metric, session_number)
Run it: DATABASE_URL=... PYTHONPATH=src:api .venv/bin/python3.14 scripts/24_migrate_cohort_response_curves.py

3. scripts/25_compute_cohort_response_curves.py – Computation script. For each grp_* flag in cohorts table, compute the response curves.
Run a SQL query using PERCENTILE_CONT(0.25/0.50/0.75) WITHIN GROUP (ORDER BY metric_value) to get the p25/p50/p75 values.
Upsert results into cohort_response_curves with ON CONFLICT (grp_flag, metric, session_number) DO UPDATE SET p25=excluded.p25, p50=excluded.p50, p75=excluded.p75, n=excluded.n

4. scripts/18_scheduled_retrain.py – After the hot-reload call at the end of the script, also call the compute_cohort_response_curves script.

5. api/routers/cohorts.py – New file. Add GET /api/cohort/{grp_flag}/response_curves endpoint.
{ grp_flag: str, curves: [{ metric: str, points: [{ session_number, p25, p50, p75, n }]}]}
Query the cohort_response_curves table filtered by grp_flag.

6. `api/main.py` – Import and register cohorts router: `app.include_router(cohorts_router, p`
7. `web/lib/types.ts` – Add `ResponseCurvePoint`, `ResponseCurve`, and `CohortCurves` types.
8. `web/lib/api.ts` – Add `getCohortResponseCurves(grpFlag: string)` function.
9. `web/components/clinical/MetricChart.tsx` – When a patient's primary `grp_*` flag is known,
10. `tests/` – Add tests:
 - Test GET `/api/cohort/grp_frailty_sarcopenia/response_curves` returns 200 and curves a
 - Test response has correct structure (`grp_flag`, curves with points)
 - Test migration is idempotent (CREATE TABLE IF NOT EXISTS)

VERIFY: `PYTHONPATH=src:api .venv/bin/pytest tests/ -v` – all tests pass.

COMMIT and leave on `feat/cohort-response-curves`.

--- END PROMPT ---

Feature 12 — Admin Dashboard for Model Health

--- START PROMPT ---

You are working on QTX-AH, a clinical intelligence platform for rehabilitation patients and

CURRENT STATE: `api/routers/admin.py` has `POST /api/admin/reload-models`. There is no `GET` endpoint.

TASK: Implement Feature 12 – Admin Dashboard for Model Health.

BRANCH: Create and work on branch `feat/admin-dashboard`. Branch from `main`.

WHAT TO DO:

- `api/routers/admin.py` – Add `GET /api/admin/model_status` endpoint (requires `X-Admin-Key`)
 - Read `retrain_state.json` from project root (use `Path(__file__).resolve().parents[2] / 'retrain_state.json'`)
 - List all `.joblib` files in the `models/` directory with their `size_mb` and `modified_at` (ISO 8601)
 - Return `{ models: {filename: {size_mb, modified_at}}, retrain_state: {...}, db_ready: bool }`Import and reuse the existing `_verify_admin_key()` helper (or equivalent) already in `admin.py`
- `api/routers/admin.py` – Add `POST /api/admin/trigger_retrain` endpoint (requires `X-Admin-Key`)
 - Accept `BackgroundTasks` parameter
 - Add a task that subprocess-runs `scripts/18_scheduled_retrain.py` from the project root
 - Return immediately: `{ status: "retrain_scheduled" }`
- `api/main.py` – Verify `admin` router is already imported and registered (it is, at line 18)
- `web/components/pages/AdminPage.tsx` – New page component:
 - Admin key input (type=password, stored in `sessionStorage` as `"qtx_admin_key"`). All API calls require this key
 - On load: fetch `GET /api/admin/model_status` and `GET /api/calibration` (existing endpoint)
 - Render three sections:
 - "Model Files" table: `filename | size | last modified`
 - "Last Retrain" panel: `timestamp, training_set_n, CV metrics (RMSE, MAE, R2, AUC-ROC)`
 - "Calibration Health" table: re-render the cohort drift data from `ModelHealthCard` component
 - Two action buttons: "Trigger Retrain" (calls `POST /api/admin/trigger_retrain`, disabled if retrain is in progress)
 - Show loading states and error messages when API calls fail
- `web/app/admin/page.tsx` – Create the Next.js page that renders `<AdminPage />`.
- `web/components/App.tsx` or `Sidebar.tsx` – Add a gear icon nav link to `/admin` at the bottom of the sidebar
- `web/lib/api.ts` – Add `getModelStatus(adminKey: string)` and `triggerRetrain(adminKey: string)`

8. tests/test_admin_auth.py (existing) – Add:

- Test GET /api/admin/model_status returns 401 without correct key
- Test GET /api/admin/model_status returns 200 with correct key and includes models and
- Test POST /api/admin/trigger_retrain schedules background task (mock BackgroundTasks)

VERIFY: PYTHONPATH=src:api .venv/bin/pytest tests/ -v – all tests pass.

COMMIT and leave on feat/admin-dashboard.

--- END PROMPT ---

Feature 13 — Wire POST /api/import/file to Real Pipeline

--- START PROMPT ---

You are working on QTX-AH, a clinical intelligence platform for rehabilitation patients and

CURRENT STATE: POST /api/import/file in api/routers/import_data.py accepts a file upload but

TASK: Implement Feature 13 – Wire POST /api/import/file to the real pipeline.

BRANCH: Create and work on branch feat/import-pipeline. Branch from main.

WHAT TO DO:

1. Read api/routers/import_data.py to understand the current stub implementation.
2. Read scripts/01_ingest.py through scripts/07_export_dashboard_data.py to understand each
3. api/routers/import_data.py – Replace the stub with a real implementation:
 - a. Save the uploaded file to a temp path using tempfile.NamedTemporaryFile
 - b. Import and call each pipeline stage in sequence using the exact function signatures
 - qtx.io.load (or equivalent) → load the Excel/CSV into a raw DataFrame
 - qtx.clean.pipeline → clean
 - qtx.phenotype.tagger → tag/classify phenotypes
 - qtx.outcomes.pipeline → compute outcomes and change scores
 - qtx.features.builder → build the feature matrix
 - c. Pass the resulting DataFrame to IngestPipeline(db).upsert(df)
 - d. db.commit()
 - e. Return { inserted: int, updated: int, errors: int, filename: str }
 - f. Always clean up the temp file in a finally block
 - g. If any pipeline stage raises, return HTTP 422 with the error message (do not let it
4. Handle both .xlsx and .csv uploads. Check file extension and route to the right loader.
5. tests/test_import_api.py (existing) – Add:
 - Test that uploading a minimal valid Excel/CSV fixture returns 200 with inserted > 0
 - Test that uploading a malformed file returns 422 with an error message
 - Test that the temp file is cleaned up after both success and failure

VERIFY: PYTHONPATH=src:api .venv/bin/pytest tests/ -v – all tests pass.

COMMIT and leave on feat/import-pipeline.

--- END PROMPT ---

Feature 14 — Retire Streamlit Dashboard

--- START PROMPT ---

You are working on QTX-AH, a clinical intelligence platform for rehabilitation patients and

CURRENT STATE: dashboard/app.py and dashboard/app_external.py exist from before the API-d

TASK: Implement Feature 14 – Retire Streamlit Dashboard.

BRANCH: Create and work on branch feat/retire-streamlit. Branch from main.

WHAT TO DO:

1. First, verify nothing in the active codebase (api/, web/, src/, scripts/, tests/) imports dashboard.
`grep -r "from dashboard" . --include="*.py" && grep -r "import dashboard" . --include="*.py"`
2. If no active dependencies found, delete the dashboard/ directory entirely.
3. Makefile – Remove the `dashboard` target (and any phony declaration referencing it).
4. README.md – Remove the Streamlit setup instructions section. The canonical UI is now the API.
5. HANDOFF.md – Remove any references to `dashboard/app.py` or `make dashboard`.
6. If dashboard/ contained any logic not present in the API/frontend (custom visualisation, etc), move it to the API/frontend.

VERIFY: PYTHONPATH=src:api .venv/bin/pytest tests/ -v – all tests pass. Confirm `make dev` works.

COMMIT and leave on feat/retire-streamlit.

--- END PROMPT ---

Feature 15 — Dosage and Dropout Calibration Tracking

--- START PROMPT ---

You are working on QTX-AH, a clinical intelligence platform for rehabilitation patients and

CURRENT STATE: CalibrationService tracks per-cohort MAE drift for the regression model on

TASK: Implement Feature 15 – Dosage and Dropout Calibration Tracking.

BRANCH: Create and work on branch feat/model-auc-tracking. Branch from main.

WHAT TO DO:

1. Read `api/services/calibration.py` fully to understand `CalibrationService.get_report()` s
2. `api/services/calibration.py` – Add two class methods:
 - a. `_compute_current_classifier_auc(cls, db) -> float | None`:
 - Query `session_predictions.responder_probability` (predicted) joined to `sessions.ove`
 - If fewer than 20 paired rows, return `None`
 - Compute AUC-ROC using `sklearn.metrics.roc_auc_score` and return it
 - b. `_compute_current_dropout_auc(cls, db) -> float | None`:
 - Same but using `session_predictions.dropout_probability` vs `sessions.is_dropout`
3. `api/services/calibration.py` – In `get_report()`, after building the cohort MAE list, appen
[
 { "model": "classifier", "baseline_auc": float|None, "current_auc": float|None, "drift_
 { "model": "dropout", ... }
]
Status thresholds for AUC drift $(\text{current} - \text{baseline}) / \text{baseline} * 100$:
 - NO_BASELINE: `baseline_auc` is `None` in `retrain_state.json`
 - OK: `drift > -5%`
 - WARNING: $-10\% < \text{drift} \leq -5\%$
 - ALERT: `drift <= -10%`(Positive drift = model improved, always OK)
4. `scripts/18_scheduled_retrain.py` – In `_retrain_classifier()` and `_retrain_dropout()` (which
 `state["classifier_auc_baseline"] = float(mean_cv_auc_classifier)`
 `state["dropout_auc_baseline"] = float(mean_cv_auc_dropout)`
 `_write_state(state)`
If these retrain functions don't exist yet, add them following the same pattern as `_re`

5. web/components/clinical/ModelHealthCard.tsx – After the existing cohort MAE table, add
6. tests/test_calibration_service.py (existing) – Add:
- Test `_compute_current_classifier_auc` returns None when <20 paired rows
 - Test AUC drift status is OK at drift=-3%, WARNING at -7%, ALERT at -12%
 - Test `get_report()` response includes `model_auc_drift` key with classifier and dropout error
 - Test `drift_pct` is None when `current_auc` is None (insufficient data)

VERIFY: `PYTHONPATH=src:api .venv/bin/pytest tests/ -v` – all tests pass.

COMMIT and leave on feat/model-auc-tracking.

--- END PROMPT ---

Audit Agent Prompt

This prompt is for a separate Claude Code session whose job is to review, verify, and merge completed feature branches. Run it after one or more feature branches have been built. It will check each branch for correctness, run tests, and merge passing branches to main.

--- START AUDIT PROMPT ---

You are the QTX-AH integration auditor. Your job is to review feature branches, verify each

CODEBASE: /Users/reetmitra/Desktop/QTX/quantumtx-ah

MAIN BRANCH: main

TEST COMMAND: PYTHONPATH=src:api .venv/bin/pytest tests/ -v

BASELINE: 483 tests passing on main before any feature work

FEATURE BRANCHES TO AUDIT (check which exist with `git branch -a`):

- feat/longitudinal-regression → Feature 1: Retrain Regression Model
- feat/bayesian-residual-correction → Feature 2: Per-Patient Bias Correction
- feat/anomaly-detection → Feature 3: Anomaly Detection (extend existing)
- feat/treatment-plan → Feature 4: Treatment Plan Generation
- feat/cohort-percentiles → Feature 5: Cohort Percentile Ranks
- feat/wearable-cadence → Feature 6: Wearable Cadence in Fall Risk
- feat/secrets-hardening → Feature 7: Secrets Rotation
- feat/cors-hardening → Feature 8: CORS Hardening
- feat/background-tasks → Feature 9: Background Task Queue
- feat/session-prep → Feature 10: Session Preparation Workflow
- feat/cohort-response-curves → Feature 11: Cohort Response Curves
- feat/admin-dashboard → Feature 12: Admin Dashboard
- feat/import-pipeline → Feature 13: Import File Pipeline
- feat/retire-streamlit → Feature 14: Retire Streamlit
- feat/model-auc-tracking → Feature 15: Model AUC Tracking

FOR EACH EXISTING BRANCH, FOLLOW THIS EXACT PROCESS:

STEP 1 – CHECKOUT AND DIFF

git checkout feat/<branch-name>

git diff main...HEAD --stat

Review the list of changed files. Confirm they match what the feature requires.

STEP 2 – CODE REVIEW (read the changed files)

For each changed file, read it and verify:

- a. The implementation matches the spec in IMPLEMENTATION.md (read it at the project root)
- b. No obvious bugs (off-by-one errors, missing null checks, wrong SQL logic)
- c. No security regressions (no new hardcoded secrets, no SQL injection via raw string con
- d. New functions have appropriate error handling (try/except with logging, not bare except
- e. ORM changes have corresponding migration scripts

STEP 3 – RUN TESTS

`PYTHONPATH=src:api .venv/bin/pytest tests/ -v 2>&1 | tail -30`

Confirm: (a) no new test failures, (b) new tests added for this feature pass, (c) total tests

STEP 4 – FUNCTIONAL SPOT-CHECK

For backend-only features: verify the key endpoint exists with:

`grep -r "router\.(get|post|put|delete)" api/routers/ | grep "<expected-path>"`

For migration scripts: verify they contain IF NOT EXISTS (idempotent)

For frontend features: read the component and verify the API call and render logic are working

STEP 5 – VERDICT

PASS: All files match spec, tests pass, no security issues → proceed to merge

FAIL: Document specific issues found → do NOT merge, output a list of what must be fixed

STEP 6 – MERGE PASSING BRANCHES

If verdict is PASS:

`git checkout main`

`git merge feat/<branch-name> --no-ff -m "merge: feat/<branch-name> - <one-line summary>"`

Verify tests still pass on main after the merge:

`PYTHONPATH=src:api .venv/bin/pytest tests/ -v 2>&1 | tail -10`

STEP 7 – HANDLE MERGE CONFLICTS

If a merge conflict occurs:

- Run `git status` to identify conflicted files
- Read both sides of the conflict
- Resolve by keeping BOTH features when possible (not choosing one over the other)
- If the conflict is in shared code (`api/main.py` router imports, `api/models/clinical.py`)
- After resolving: `git add <files> && git commit -m "merge: resolve conflict between feat/<branch-name>"`
- Run tests again to confirm resolution is correct

DEPENDENCY ORDER (merge in this order to minimise conflicts):

- feat/secrets-hardening (no code deps – only config files)
- feat/cors-hardening (only `api/main.py`)
- feat/longitudinal-regression (only `models.yaml` and scripts)
- feat/background-tasks (api/services only)
- feat/bayesian-residual-correction (api/services/prediction.py + migration)
- feat/anomaly-detection (extends existing `api/services/anomaly.py`)
- feat/wearable-cadence (extends `anomaly.py`)
- feat/treatment-plan (new files: `plan.py`, `PlanCard.tsx`)

9. feat/cohort-percentiles

(extends benchmark.py + new frontend component)
10. feat/session-prep

(extends ask.py + QAPanel.tsx)
11. feat/model-auc-tracking

(extends calibration.py + retrain script)
12. feat/admin-dashboard

(new files: admin page, model_status endpoint)
13. feat/cohort-response-curves

(new table, new script, extends MetricChart.tsx)
14. feat/import-pipeline

(extends import_data.py)
15. feat/retire-streamlit

(deletions only – do last)

AFTER ALL MERGES:

Run the full test suite one final time:

```
PYTHONPATH=src:api .venv/bin/pytest tests/ -v
```

Report the final test count and any failures.

Output a summary table:

Feature	Branch	Status	Tests Added	Merged
1. Longitudinal Regression	feat/longitudinal-regression	PASS/FAIL	N	YES/NO
...				

CRITICAL RULES:

- Never force push to main
 - Never skip tests before merging
 - Never merge a branch with failing tests
 - If you are unsure whether a conflict resolution is correct, STOP and describe the conflict
 - Do not delete feature branches after merging (leave for reference)
 - If a feature branch does not exist, skip it and note it in the summary table as "NOT BU"
- END AUDIT PROMPT ---

Quick Reference

#	Feature	Branch	Priority
7	Secrets Rotation	feat/secrets-hardening	P0 – blocks go-live
8	CORS Hardening	feat/cors-hardening	P0 – blocks go-live

#	Feature	Branch	Priority
1	Regression Retrain	feat/longitudinal-regression	P1 — credibility
3	Anomaly Detection	feat/anomaly-detection	P1 — clinical value
4	Treatment Plan	feat/treatment-plan	P1 — clinical value
2	Bayesian Correction	feat/bayesian-residual-correction	P2
5	Cohort Percentiles	feat/cohort-percentiles	P2
9	Background Tasks	feat/background-tasks	P2
10	Session Prep	feat/session-prep	P2
15	AUC Tracking	feat/model-auc-tracking	P3
11	Response Curves	feat/cohort-response-curves	P3
12	Admin Dashboard	feat/admin-dashboard	P3
6	Wearable Cadence	feat/wearable-cadence	P3
13	Import Pipeline	feat/import-pipeline	P3
14	Retire Streamlit	feat/retire-streamlit	P4 — cleanup last